

routes/dept.ts

```
...  
router.get('/add/:code', async function(req: Request, res: Response, next: NextFunction)  
{  
  const dept = new deptmodel.Dept(req.params.code);  
  await deptmodel.insertMany([dept]);  
  res.json({ code: dept.code });  
});  
...
```

- req.params.code grabs the :code from the URL, e.g. /dept/add/hr → "hr"
- Creates a new Dept object with that code
- insertMany([dept]) saves it to MongoDB (wrapped in an array because the function expects a list)
- res.json(...) sends it back as JSON

Q: Why is it insertMany and not insertOne or createCollection?

A: insertMany function in models/dept.ts accepts an array, so we pass [dept] — an array with one item. In a real app you'd probably have both insertOne and insertMany for different use cases.

MongoDB creates collections automatically the first time you insert a document into them. So when you call insertMany on the dept collection for the first time, MongoDB creates the dept collection on the spot — you don't need to explicitly create it first. createCollection exists but is only needed in special cases like when you want to set specific options upfront (e.g. capped collections, schema validation).

Q: Why is it res.json and not res.send?

A: res.json() automatically sets the Content-Type header to application/json and serializes the object properly. res.send() with an object would also work, but res.json() is more explicit and conventional when you're returning JSON data.

You could technically keep res.send({ code: dept.code }) and it would work the same way since Express detects objects and handles them like JSON — but res.json() makes your intent clearer.

```
...  
router.get('/all/withstaff/', async function(req: Request, res: Response, next:  
NextFunction) {  
  const depts = await deptmodel.all();  
  const staffs = await staffmodel.all();  
  const result = depts.map(d => {  
    code: d.code,  
    staffs: staffs.filter(s => s.dept === d.code)  
  });  
  res.json(result);  
});
```

```
});  
` ``
```

- Fetches all departments and all staff from MongoDB
- For each department, filters the staff list to only those whose dept matches that department's code
- Builds a new object combining both — { code: "hr", staffs: [...] }
- Note this route is above /all/ intentionally — if it were below, Express would match /all/withstaff as /all/ with withstaff never reached

```
` ``
```

```
router.get('/all/', async function(req: Request, res: Response, next: NextFunction) {  
  const depts = await deptmodel.all();  
  res.json(depts);  
});  
` ``
```

- Calls deptmodel.all() which does collection.find({}) — fetches every document in the dept collection
- Returns them as a JSON array

Q: How does the deptmodel.all() or staffmodel.all() work/ use collection.find({})?

A: This is the underlying code below:

```
` ``
```

```
export async function all(): Promise<Dept[]> {  
  const depts = await find({});  
  return depts;  
}  
` ``
```

Which has:

```
` ``
```

```
export async function find(p: Filter<Document>): Promise<Dept[]> {  
  const collection = db.collection('dept');  
  const cursor = collection.find(p);    // asks MongoDB for all documents  
  const depts: Dept[] = [];  
  while (await cursor.hasNext()) {    // loop through results one by one  
    const dbobj = await cursor.next(); // get next raw document from MongoDB  
    if (dbobj) {  
      depts.push(new Dept(dbobj.code)); // convert to Dept object  
    }  
  }  
  return depts;  
}  
` ``
```

So the flow is:

all()

- find({})
- MongoDB: "give me everything in the dept collection"
- loop through raw documents
- convert each one into a Dept object
- return the array

Which is similar to what we have done in mongosh with `` `` `var cursor = db.message.find()`` ```

Q: What is `dbobj.code` in the following code segment?

```
`` `while (await cursor.hasNext()) {    // loop through results one by one
  const dbobj = await cursor.next(); // get next raw document from MongoDB
  if (dbobj) {
    depts.push(new Dept(dboj.code)); // convert to Dept object
  }
} `` `
```

A: `dbobj` is the raw document that comes back from MongoDB. `dbobj.code` just reads the `code` property from that object — so you get "hr".

When you do `new Dept(dboj.code)`, the `Dept` constructor only accepts `code`. So the new `Dept` object only ever has `code` on it. `_id` isn't passed in, so it never gets stored on the object — it's not silently dropped so much as it was never picked up in the first place.

Think of it like unpacking a box — `dbobj` is the full box containing `_id` and `code`, but you only take out `code` and put it into the `Dept` object. The `_id` just stays in the box and gets discarded.

routes/staff.ts

```
`` `
router.get('/add/:id/:name/:code', async function(req: Request, res: Response, next:
NextFunction) {
  const staff = new staffmodel.Staff(req.params.id, req.params.name,
req.params.code);
  await staffmodel.insertMany([staff]);
  res.json({ id: staff.id, name: staff.name, dept: staff.dept });
});
`` `
```

- `req.params.id`, `req.params.name`, `req.params.code` grab the three parts from the URL, e.g. `/staff/add/1/aaron/hr` → "1", "aaron", "hr"
- Creates a new `Staff` object with those three values
- `insertMany([staff])` saves it to MongoDB
- `res.json(...)` sends back the `staff` object as JSON — note we manually pick `id`, `name`, `dept` to avoid sending MongoDB's internal `_id` field back to the user

Q: So in order to specify the two ways of controlling your output, there are two ways, one is creating a new object and the other is to specify the JSON?

A: Yes. In the dept code, we created a filter, using a new object (like a class) with the fields that we want. In the staff code, we manually specified the fields we want in `res.json()`. Both achieve the same result. The class approach is cleaner when you're reusing the same shape of data across multiple places in your app. The manual `res.json()` approach is quicker for one-off responses.

```

```
router.get('/all/', async function(req: Request, res: Response, next: NextFunction) {
 const staffs = await staffmodel.all();
 res.json(staffs);
});
```

```

- Calls `staffmodel.all()` which does `collection.find({})` — fetches every document in the staff collection
- Returns them as a JSON array